

On the Influence of Refactoring Types on Merge Effort

André Oliveira, João Victor Monteiro, Vânia Neves, Alexandre Plastino, Bianca Trinkenreich, Alessandro Garcia *Member, IEEE*, and Leonardo Murta

Abstract—Modern software development involves parallel work and concurrent changes, requiring code merging. Prior studies report that 10% to 20% of merge attempts result in conflicts, often requiring manual intervention. The literature explores factors that generate conflicts, including refactorings, but does not analyze how individual refactoring types influence the manual effort required to resolve them. We analyzed 64 open-source Java projects and applied association rule mining to measure the strength of associations between specific refactoring types and merge effort. Our results show that refactoring types relate to merge effort with varying strength. In particular, *Rename Attribute*, *Move Class*, *Extract Variable*, *Change Return Type*, and *Split Parameter* exhibit some of the strongest associations, especially when a higher number of such refactorings is present in the merge branches. We also find that both the number of refactorings and their diversity independently increase merge effort, both in terms of occurrence and intensity. Additionally, the co-occurrence of refactorings across parallel branches is associated with higher merge effort, particularly when combining structural transformations with changes to method signatures and data-structure representations, whereas more localized changes are less frequent in the most impactful combinations.

Index Terms—Software Merge, Merge Effort, Refactoring, Association Rules Extraction, Data Mining

I. INTRODUCTION

During software development, teams make concurrent modifications. They must merge these changes, a challenging process [1]. Studies report 10%–20% of merges failing [2], [3], with some projects approaching 50% [2], [4]. Merging parallel changes requires considerable effort because teams must resolve conflicts and therefore decrease software productivity. Researchers have proposed multiple techniques to resolve merge conflicts [5], [6], including automated [7], [8], [9] and semi-automated methods [5], [10], [6]. However, developers often resolve conflicts manually when tools fail [1]. In any case, given the limited accuracy of existing tools, developers still need to verify merge resolutions manually.

Developers modify the source code for several well-established reasons. They fix bugs (corrective maintenance), adapt the system to new environments (adaptive maintenance), improve performance, testability, security, and usability (perfective maintenance), and prepare the system for fu-

ture changes (preventive maintenance) [11], [12]. Permeating these types of code changes, we have refactorings that help manage software evolution while preserving design quality. They restructure code without changing behavior, improving modularity and maintainability [13], [14], [15]. Depending on volume and type, refactorings can affect many code regions. Developers must weigh short-term impacts, as unmanaged refactorings may overlap with parallel changes and trigger merge conflicts.

To illustrate how refactorings contribute to merge effort, we present a real example from GitHub that shows a small excerpt of the *diff*. Code 1 shows a scenario from merge commit 77e002a in the *byte-buddy* repository¹, obtained by the `git diff 77e002a 77e002a^1 77e002a^2` command. Here, 77e002a^1 and 77e002a^2 denote the first and second parents of the merge commit, respectively. The *diff* uses a unified format where “-” and “+” denote removals and additions in the branches, respectively, while “--” and “++” indicate edits introduced during merge resolution. In this case, branch 1 applied a *Change Variable Type* refactoring by introducing `MethodList<?>`, whereas branch 2 performed a *Rename Attribute* refactoring, shortening `targetMethodCandidates` to `targetCandidates`. The merge combines both changes, yielding `MethodList<?> targetCandidates`. The presence of a “++” line corresponds to one unit of merge effort (*effort* = 1). This example highlights that even simple and seemingly independent refactorings, when applied concurrently, may require manual intervention during merging, motivating the need to better understand how different refactoring types contribute to merge effort.

Code 1 Example of merge effort identified in commit 77e002a of the *byte-buddy* repository.

```
- private final MethodList<?> targetMethodCandidates;
- private final MethodList targetCandidates;
++ private final MethodList<?> targetCandidates;
```

Prior studies have analyzed the relationship between refactorings and code merging [16], [17], [18], [19], [20]. Early work proposed tools to detect common refactorings, such as renaming and moving code, before merges to support developers’ decisions [16], [19], [17]. Mahmoudi and Nadi [18] investigated frequent refactoring types and the potential for automated merging, while Mahmoudi et al. [20] found that

André Oliveira, Vânia Neves, Alexandre Plastino, and Leonardo Murta are with the Computing Institute, Fluminense Federal University, Niterói, RJ, Brazil. E-mail: {andrelucio, vania, plastino, leomurta}@ic.uff.br. Bianca Trinkenreich is with the Colorado State University, Fort Collins, United States of America. E-mail: bianca.trinkenreich@colostate.edu. Alessandro Garcia is with the Pontifical Catholic University of Rio de Janeiro (PUC-Rio), Rio de Janeiro, Brazil. E-mail: afgarcia@inf.puc-rio.br. João Victor Monteiro is with the Veiga de Almeida University. E-mail: jjoaovictormonteiro@gmail.com.

¹<https://github.com/raphw/byte-buddy>

22% of merge conflicts involve refactorings and that 11% of conflicting regions contain at least one refactoring, which makes such conflicts more complex to resolve. However, these studies did not examine which refactoring types are more likely to increase the frequency and intensity of manual effort required to resolve merge conflicts, nor whether some refactoring types are (in)compatible to occur in parallel, across branches. Moreover, they considered a limited subset of refactorings compared to the catalog described by Fowler [14].

This paper investigates the influence of individual refactoring types on the manual effort required to resolve merge conflicts, referred to as *merge effort*. We analyze 64 open-source Java projects hosted on GitHub, covering 33 distinct refactoring types, and apply association rule mining [21] to extract co-occurrence patterns between these types and the computed merge effort. We adopt code churn as a surrogate for effort, following Sjøberg et al. [22], who reported a moderate to strong correlation between churn and maintenance effort.

This study extends our previous work published at ICSE 2023 [23]. In that study, we investigated how the overall number of refactorings and their distribution across merge branches influence merge effort. The results showed that merges involving refactorings occurred with a 24% higher frequency of merge effort, and that concurrent refactorings in both branches were substantially more frequently associated with high-intensity effort. However, that study analyzed refactorings only at an aggregated level, without distinguishing the specific types of refactoring operations involved. In this paper, we extend that analysis by investigating the roles of individual refactoring types and their interactions, and we use RefactoringMiner v3.1, which offers much higher accuracy for refactoring detection than the version used in our prior study. We consider a wide range of 33 refactoring types and analyze how their occurrence, quantity, diversity, and combinations across branches are associated with merge effort. This type-level analysis provides a more fine-grained understanding of how different refactoring operations influence merge effort during code integration.

To investigate the influence of refactoring types on merge effort, we address the following research questions, detailed in Section III:

RQ1: Which refactoring types most influence the occurrence of merge effort?

Understanding how different refactoring types influence merge effort is essential for improving collaborative development reliability. This RQ examines whether some refactoring types are more strongly associated with merge effort than others.

RQ2: Does the amount of different refactoring types influence the merge effort?

The analysis in RQ1 considers just the occurrence or not of each refactoring type. However, for the cases where a given refactoring type occurs, the amount of refactorings of such type may also reveal new risk factors in code integration.

RQ3: How do combinations of refactoring types, applied concurrently in different branches, influence the merge effort?

Refactorings implemented in parallel across branches can interact in complex ways, particularly when they affect related

elements (e.g., method signatures, inheritance, and interface implementations), leading to overlapping changes that hinder automatic conflict resolution and increase integration effort.

Our investigation reveals that, in our dataset, most refactoring types are associated with higher-effort merges, whereas merges without refactorings show lower rates of high-effort cases. Notably, *Rename Attribute* is linked to a 400% increase in merge effort, while structural refactorings such as *Move Class* reach even stronger associations (+431%). Other transformations, including *Extract Variable* (+367%) and *Change Return Type* (+320%), follow similar patterns, and even less frequent operations, such as *Split Parameter*, show substantial associations with merge effort. We also find that both refactoring diversity and volume are independently associated with merge effort, affecting not only its occurrence but also its intensity. Finally, the co-occurrence of refactoring types across parallel branches shows stronger associations with merge effort than isolated transformations, particularly when combining structural changes with modifications to method signatures or data representations, whereas more localized changes appear less frequently among the most impactful combinations.

Our findings suggest that developers should avoid the concurrent application of structural and high-risk refactoring combinations across branches, favoring early integration, smaller increments, and short-lived branches. Monitoring refactoring volume and diversity can help anticipate integration risks, while tool support (e.g., CI bots, GitHub Actions, and Gerrit hooks) can proactively flag risky scenarios and guide merge timing decisions. Section III-D discusses these implications.

The remainder of this paper is organized as follows. Section II describes the research process, dataset, and techniques. Section III presents the results and answers the research questions. Section IV outlines the threats to validity. Section V reviews related work. Section VI summarizes our contributions and future research directions.

II. MATERIALS AND METHODS

Our research process comprises three phases (Figure 1). Phase one defines the project selection criteria (Section II-A). Phase two collects refactoring and merge data and computes merge effort measures, composing the dataset (Section II-B). Phase three (Section II-C) applies association rule mining [21] to address the research questions (Section I), uncovering patterns that may be overlooked by manual analysis. The data, scripts, and reproducibility guidelines are available in the replication package². Data extraction and mining were conducted between May and July 2025. The dataset spans from August 2008 to July 2025, with an average project history of 12.8 years (range: 4.62–16.90 years).

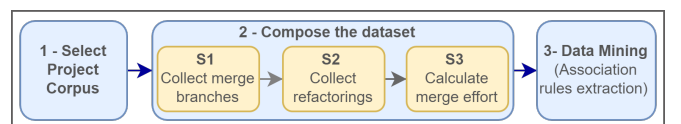


Fig. 1. The phases of the research process.

²<https://github.com/gems-uff/refactoring-merge>

A. Select Project Corpus

We constructed the corpus from mature, relevant open-source GitHub projects using the GraphQL API (v4)³. We identified public repositories that met these criteria: (i) not forks, (ii) $\geq 5,000$ stars, (iii) not archived, and (iv) received at least one push in the last three months. Following Kalliamvakou et al. [24], we excluded forks to ensure single project representation. We verified that no forked repositories met the selection criteria. The 5,000-star threshold ensured corpus relevance and widely recognized repositories [25]. Excluding archived and inactive repositories ensured ongoing development. The search resulted in a final set of 6,161 repositories.

After initial selection, we further filtered the 6,161 repositories by analyzing the contributors and the commits metadata. We retained repositories with (i) ≥ 10 contributors and (ii) $\geq 3,000$ commits in the default branch. The contributor filter excluded personal or coursework projects, ensuring collaborative, production-level software [24]. The commit threshold eliminated immature or short-term projects, focusing on sustained development. The contributor filter yielded 5,714 repositories. Of these, 1,989 met the $\geq 3,000$ commit criterion.

From the 1,989 repositories, we selected only Java-primary projects, as reported by GitHub. We focused on Java due to several factors: (i) Java's popularity (TIOBE index [26], Stack Overflow Developer Survey [27]). Although Java is not the most popular language overall, its consistent ranking among the top positions demonstrates its enduring relevance and influence within the software development community; (ii) the need to ensure consistency and comparability with our prior work [23], which was also conducted on Java systems; (iii) the reduced confounding effects associated with language-specific characteristics, such as differences in syntax, refactoring practices, and development ecosystems; and (iv) the suitability of Java for refactoring analysis, given its strong object-oriented nature, mature tooling support, and large availability of long-lived and well-maintained open-source repositories. These filters yielded 112 repositories.

We manually inspected the 112 repositories via GitHub pages and, when available, their official websites. The inspection removed non-software projects or those with non-English documentation. One repository contained only documentation (e.g., books, manuals), explaining its classification; we removed it, leaving 111 projects. We focused on English documentation for complete artifact understanding. We excluded seven repositories with primarily Chinese documentation, as inconsistent translation quality risked the accuracy and reliability of the analysis. The exclusions resulted in 104 repositories.

Our study considers merge commits resulting from standard `git merge` operations that integrate *named branches*, including those associated with *pull request* integrations, as well as merges created within pull request branches prior to integration. We identify a commit as a merge commit if it has exactly two parents in the Git history. Additionally, we considered only non-fast-forward merge commits as valid, since we aimed to assess merging effort. A fast-forward merge

moves the target branch pointer to the source branch without creating a new merge commit, provided there is a direct linear relationship between these two branches. In contrast, `git merge --no-ff` performs a non-fast-forward merge, creating a new merge commit and preserving history and branch structure even when a fast-forward merge is possible. We calculated the distribution of valid merge commits per project. Using Tukey's fences [28], defined by the lower bound ($Q1 - 1.5 \times IQR$) and upper bound ($Q3 + 1.5 \times IQR$), we analyzed the distribution of valid merge commits to establish selection criteria and filter out extreme cases.

To ensure a homogeneous dataset, we excluded projects based on the distribution of valid merge commits: 13 projects exceeding the upper threshold (5,503 merges) and 27 projects below the first quartile (218 merges) were removed. The final corpus comprises 64 systems, a scale consistent with or exceeding related studies (e.g., 10–70 projects) [29], [30], [31]. We further discarded 28.3% of `--no-ff` merges and excluded octopus merges (more than 2 parents), which are inherently conflict-free. This resulted in 91,270 merge commits.

These criteria filter trivial cases without biasing the dataset toward high-effort outcomes. By not constraining projects based on refactoring frequency, the corpus reflects a natural distribution of real-world development practices. This allows for an unbiased assessment of refactoring influence across diverse merge scenarios. Detailed project-level information is available in our replication repository².

B. Compose the Dataset

As illustrated in Figure 1, the second phase was carried out in three steps, each implemented in a separate Python script: (S1) identifying the commits that comprise each branch of the merge commits, (S2) collecting the refactorings implemented within the commits of each branch, and (S3) calculating the merge effort.

Script S1 collected all commits from the branches linked to the merge commits found in the repositories. Figure 2 shows an example of version history with two merge commits (C10, C12) highlighted in red. For C10, the script identifies `branch_1C10` contains C7, while `branch_2C10` includes C6 and C8. Since Script S1 handles nested merges, for C12, it identifies `branch_1C12` contains C3, C5, C9. Branch_2_{C12} includes C2, C4, C6, C7, C8, C11, encompassing all commits from C10's branches. We used Python's `pygit2`⁴ to gather project versioning data.

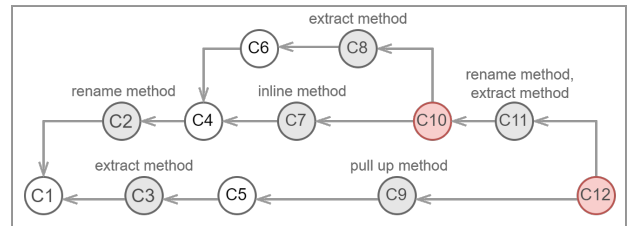


Fig. 2. Implementing refactorings on merge commit branches.

Script S2 identified refactorings in merge commit branches. Figure 2 shows commits that contain refactorings highlighted

³<https://docs.github.com/pt/graphql>

⁴<https://www.pygit2.org/>

in gray. To detect refactorings, we used *RefactoringMiner* (v3.1, March 2026), which can detect more than 100 refactoring types. Its authors reported 99.6% precision and 99.4% recall, establishing it as state-of-the-art for automated refactoring detection [32]. After script execution, we computed the number of refactorings by type in merge commit branches. We analyzed 700,308 commits using *RefactoringMiner*.

In this study, refactorings are identified in the branches' development history prior to the merge commit. Thus, our analysis focuses on refactoring activities introduced before the integration step and excludes refactorings performed during merge resolution. We examine 33 types of refactorings, including 26 from Fowler's catalog [14]: *Change Return Type*, *Extract Attribute*, *Extract Class*, *Extract Interface*, *Extract Method*, *Extract Subclass*, *Extract Superclass*, *Extract Variable*, *Inline Method*, *Inline Variable*, *Merge Attribute*, *Move Attribute*, *Move Class*, *Move Method*, *Pull Up Attribute*, *Pull Up Method*, *Push Down Attribute*, *Push Down Method*, *Rename Attribute*, *Rename Class*, *Rename Method*, *Rename Parameter*, *Rename Variable*, *Split Attribute*, *Split Parameter*, and *Split Variable*. We also included seven additional types defined by Tsantalis et al. [33]: *Change Parameter Type*, *Change Variable Type*, *Merge Parameter*, *Merge Variable*, *Parameterize Variable*, *Replace Attribute*, and *Replace Variable with Attribute*. We selected these refactoring subsets based on recent findings indicating their frequent use by developers in practice and their exploration in recent research [34], [35], [36], [37].

Finally, **Script S3** calculated the merge effort. We investigated the relationship between refactorings and this effort using a metric defined by Prudêncio et al. [38] and implemented by Moura and Murta [39]. We computed the merge effort by calculating merge code churn via a `diff` between the base (common ancestor) and merged versions. This generates a multiset including all committed actions. We subtracted the branch-level multiset from the merge multiset. The resulting multiset contains only lines that were added or explicitly removed during the merge, and the merge effort corresponds to the total number of actions taken. The complete formulation and implementation details are available in our replication package².

Conflicts are categorized into: (i) textual, (ii) build (syntactic/static semantic), and (iii) test (behavioral) conflicts [40]. By defining merge effort as the lines added or removed during integration, our approach accounts for all three types. Unlike Git, which only detects textual conflicts, our metric captures the manual edits required to resolve build and test inconsistencies. Consequently, we avoid underestimating the actual integration effort, a common limitation in studies focused solely on textual conflicts.

We define merge effort as the effort required to reconcile concurrent changes during code integration, which differs from the effort required to perform refactorings. Our analysis focuses on how refactorings introduced in parallel branches interact during integration. Rather than measuring the effort to apply refactorings, we examine how they interfere with independent edits in another branch. For example, while a *Rename Attribute* may require updating multiple references, we do not measure the effort to apply such transformations,

but rather how they interfere with concurrent edits in another branch. Refactorings that propagate across multiple references or alter structural relationships may overlap with concurrent changes. In such cases, merge tools based primarily on textual comparison may struggle to automatically integrate these edits, requiring developers to interpret the refactoring intent and reconcile changes across different code locations. This helps explain why certain refactoring types are more frequently associated with merge effort in our dataset.

Building on this definition, we operationalize merge effort using code churn, measured as the number of added and removed lines in merge commits. This choice enables automated large-scale analysis across thousands of merge scenarios and aligns with prior work that adopts code churn as a proxy for maintenance effort and change complexity [22], [41], [42], [43], [44]. Our metric captures the additional changes required to reconcile concurrent modifications during merge resolution, rather than the total changes introduced by refactorings, thereby focusing on the integration process. Consequently, large changes do not necessarily imply high merge effort, while small changes may still require substantial reconciliation. Furthermore, VCSs such as Git operate at the line level; merge conflicts are detected and resolved line by line, making line-based churn naturally aligned with the granularity at which merge effort manifests in practice. This notion is consistent with prior work [22], which reported a moderate-to-strong correlation ($\rho = 0.59\text{--}0.66$) between code churn and maintenance effort. For example, when independent changes affect separate files or non-overlapping regions, merge effort is typically negligible. In contrast, integrating changes that involve structural modifications or overlapping edits may require substantial reconciliation effort.

While more fine-grained metrics, such as Levenshtein edit distance [45], can capture change magnitude at a lower granularity [46], they operate at a granularity that is not directly aligned with the underlying merge mechanism, and tend to be more computationally expensive, which has limited their adoption in large-scale studies [47], and do not fully capture the reconciliation effort involved in merge resolution. Time-based measures are often unreliable in open-source contexts because accurately identifying the start and end of merge-resolution activities is not feasible from repository data alone. Qualitative analysis is inherently limited in scalability and would not be applicable to the 91,270 merge commits analyzed in this study. We also acknowledge that code churn does not capture all dimensions of merge effort, such as cognitive and coordination aspects or the time required for resolution.

To further assess the validity of code churn as a proxy for merge effort, we conducted a targeted manual validation on a stratified sample of 180 merge commits with non-zero effort ($\text{Effort} > 0$), considering the three effort levels (*low*, *medium*, and *high*), and the same distribution criteria in each group, as defined in RQ1. For each case, we inspected the combined `diff` (`git diff merge_shal merge_shal^1 merge_shal^2`) to verify whether the observed churn corresponded to manual reconciliation actions, with particular attention to textual conflicts involving refactorings across branches. Overall, we identified such conflicts in 39 out of the 180 analyzed merges, distributed

as 3, 15, and 21 cases in the low-, medium-, and high-effort groups, respectively.

These conflicts involved refactoring types such as *Change Parameter Type* (22), *Rename Method* (16), *Change Return Type* (10), *Rename Class* (7), *Move Method* (4), *Rename Attribute* (4), *Extract Method* (2), *Extract Class* (2), *Move Class* (2), *Extract Interface* (1), and *Split Attribute* (1). Although this analysis is based on a sample, the presence and distribution of these refactoring types are consistent with the patterns observed in our large-scale analysis, particularly those associated with changes to method signatures, structural reorganization, and interface modifications. This alignment provides additional support for interpreting code churn as a meaningful proxy for merge effort in the context of concurrent refactorings. The examples identified during this inspection are available in our replication package².

C. Data Mining - Association Rules Extraction

We used association rule mining [21] to uncover relationships between refactoring types and merge effort. This exploratory technique identifies patterns within software repositories without predefined hypotheses, generating insights that can confirm or expand existing software engineering theories [21]. By identifying frequent associations among attributes, this approach reveals trends that might otherwise go unnoticed during manual analysis.

To investigate the relationship between refactoring and merge effort, we analyzed 33 refactoring types (represented as *refactoring_type_x* for branches *b1* and *b2*) alongside code churn as the effort metric. As summarized in Table I, our dataset comprises 67 attributes: 33 refactoring types per branch plus the merge effort indicator.

TABLE I
ATTRIBUTES CONSIDERED IN THE ANALYSIS.

Attribute	Description
<i>refactoring_type_x_{b1}</i>	number of refactoring type <i>x</i> implemented in branch 1.
<i>refactoring_type_x_{b2}</i>	number of refactoring type <i>x</i> implemented in branch 2.
<i>effort</i>	number of new lines included or excluded in the merge commit (code churn)

The method proposed in this work uses the concept of multidimensional association rules [21]. Given a relation (or table) *D*, a multidimensional association rule $X \rightarrow Y$, defined on *D*, is an implication of the form: $X_1 \wedge X_2 \wedge \dots \wedge X_n \rightarrow Y_1 \wedge Y_2 \wedge \dots \wedge Y_m$, where $n \geq 1$, $m \geq 1$, and $X_i (1 \leq i \leq n)$ as well as $Y_j (1 \leq j \leq m)$ are conditions defined in terms of the distinct attributes of *D* [48], [21].

The rule $X \rightarrow Y$ indicates, with a certain assurance, that antecedent *X* is associated with consequent *Y*. We evaluate rule relevance using three standard measures: *Support*, *Confidence*, and *Lift* [48]. *Support* captures how frequently both the antecedent and the consequent occur together in the dataset, while *Confidence* reflects how often the consequent is observed when the antecedent occurs. These two measures are used to

filter the extracted rules, and only rules that meet predefined minimum thresholds are retained for analysis.

For a straightforward illustration of *Support* and *Confidence* calculation, we refer to Table II. This table presents three columns: i) line id, ii) number of *Extract Method* implemented in branch 1; iii) number of *Rename Method* implemented in branch 2; iv) merge effort (as “true”/“false”). Each row corresponds to a unique merge commit. We discretized refactorings into four levels: zero (“0”), units (“u”), dozens (“d”), and hundreds or more (“ ≥ 100 ”). For rule *R*: $extract_method_b1 = “d” \wedge rename_method_b2 = “d” \rightarrow effort = “true”$, we identify four records in *D* meeting all three conditions (rows 1, 4, 6, 8). This results in $T_{(X \cup Y)} = 4$. Given *D* has 8 entries ($T = 8$), we calculate *Sup*(*R*) as 50% (4/8). For *Confidence*, the rule achieves 80% (4/5) since antecedent conditions ($extract_method_b1 = “d” \wedge rename_method_b2 = “d”$) hold in five entries ($T_X = 5$): rows 1, 3, 4, 6, 8.

TABLE II
REFACTORING TYPE AND MERGE EFFORT DATASET SAMPLE.

#	extract_method_b1	rename_method_b2	effort
1	“d”	“d”	“true”
2	“ ≥ 100 ”	“ ≥ 100 ”	“false”
3	“d”	“d”	“false”
4	“d”	“d”	“true”
5	“u”	“u”	“false”
6	“d”	“d”	“true”
7	“0”	“0”	“false”
8	“d”	“d”	“true”

We also consider *Lift* to assess the strength of the association between antecedent and consequent. Intuitively, *Lift* indicates how much more frequently the consequent occurs when the antecedent is present, compared to what would be expected if they were independent. A *Lift* value equal to 1 indicates independence, whereas values greater than 1 indicate a positive dependence, and values lower than 1 indicate a negative dependence.

Using rule *R* as an example, the consequent’s *Support* – *Sup*(*effort* = “true”) – in *D* is 50%, representing entries meeting *effort* = “true”. Rule *R*’s *Lift* is 1.6, calculated as $Lift(R) = 80/50 = 1.6$ (80% is rule *Confidence*). This *Lift* suggests that when dozens of *Extract Method* are in branch 1 and *Rename Method* in branch 2, the merge effort probability increases by 60%. In other words, *effort* = “true” probability in *D*, initially 50%, rises to 80% given antecedent conditions ($extract_method_b1 = “d” \wedge rename_method_b2 = “d”$).

We applied the Apriori algorithm [49] to extract association rules, using its implementation in a Python library⁵. We selected Apriori as a well-established method for frequent pattern mining, which generates candidate itemsets and prunes the search space using the anti-monotonic property of support, ensuring that infrequent itemsets do not generate supersets. This guarantees completeness with respect to the specified support and confidence thresholds. Alternative algorithms (e.g., FP-Growth [50] and Eclat [51]) produce the same rules for

⁵http://rasbt.github.io/mlxtend/user_guide/frequent_patterns/apriori/

a given dataset and thresholds, differing mainly in computational efficiency, depending on the dataset’s characteristics. As our dataset size and minimum support do not impose computational constraints, Apriori provides a simple, well-understood, and reproducible solution. We set a minimum support threshold of 20 merge commits to reduce coincidental or non-representative patterns.

For RQ3, to analyze refactoring types’ influence in parallel merge branches, we extracted association rules. We identify combinations that frequently co-occur across branches and are linked to the merge effort. These rules show two specific refactorings, each in a separate branch, increasing merge effort chances (e.g., $refac_typeX_{b1} = \text{“true”} \wedge refac_typeY_{b2} = \text{“true”} \Rightarrow effort = \text{“true”}$). To identify if concurrently applied refactorings in different branches associate with increased merge effort, we adapted rule formulation to capture refactoring *co-occurrence across branches* regardless of specific branch assignment (1 or 2). That is, we treated $refac_typeX$ in branch 1 and $refac_typeY$ in branch 2 as equivalent to the inverse. To abstract this, we redefined rule support to reflect the total merge commits where both refactorings occur across branches, independently of their implementation branch. We calculated this adjusted support by summing the occurrences of both variants and subtracting the overlap where they appeared in both branches. This aggregation allowed us to compute a single *Lift*, measuring the frequency of merge effort when two refactorings occur in parallel in branches b_α and b_β (e.g., $Sup(rule)_{new} = Sup(refac_typeX_{b_\alpha} = \text{“true”} \wedge refac_typeY_{b_\beta} = \text{“true”} \Rightarrow effort = \text{“true”})$). We applied the same logic to compute antecedent support, $Sup(ant)_{new} = Sup(refac_typeX_{b_\alpha} = \text{“true”} \wedge refac_typeY_{b_\beta} = \text{“true”})$, then calculated adjusted *Lift* as: $Lift(rule)_{new} = Sup(rule)_{new} / (Sup(ant)_{new} \times Sup(effort = \text{“true”}))$.

For completeness, the formal definitions of *Support*, *Confidence*, and *Lift* are available in our replication package².

III. RESULTS AND DISCUSSION

This section presents and analyzes the results, addressing the research questions introduced in Section I.

A. RQ1: Which Refactoring Types Most Influence the Occurrence of Merge Effort?

This RQ investigates how each refactoring type’s frequency influences merge effort, independent of the branch where it was applied. Figure 3 summarizes results. The y-axis displays the 33 analyzed refactoring types from our study, listed in alphabetical order. Initially, we discretized merge effort into a binary variable and extracted association rules for each refactoring type based on refactoring counts, categorized into four ranges: $qty_refac_type = \text{“0”}$ (zero) / “u” (units, 1–9) / “d” (dozens, 10–99) / $\text{“}\geq 100\text{”}$ (hundreds or more) $\rightarrow effort = \text{“true”}$. We used a grayscale gradient to distinguish these ranges: lightest gray for zero refactorings and darkest gray for hundreds or more. The x-axis shows each extracted rule’s *Lift* value, indicating the strength of the relationship between the antecedent (refactoring frequency) and the consequent (merge effort occurrence). We set the merge effort to “true” as we

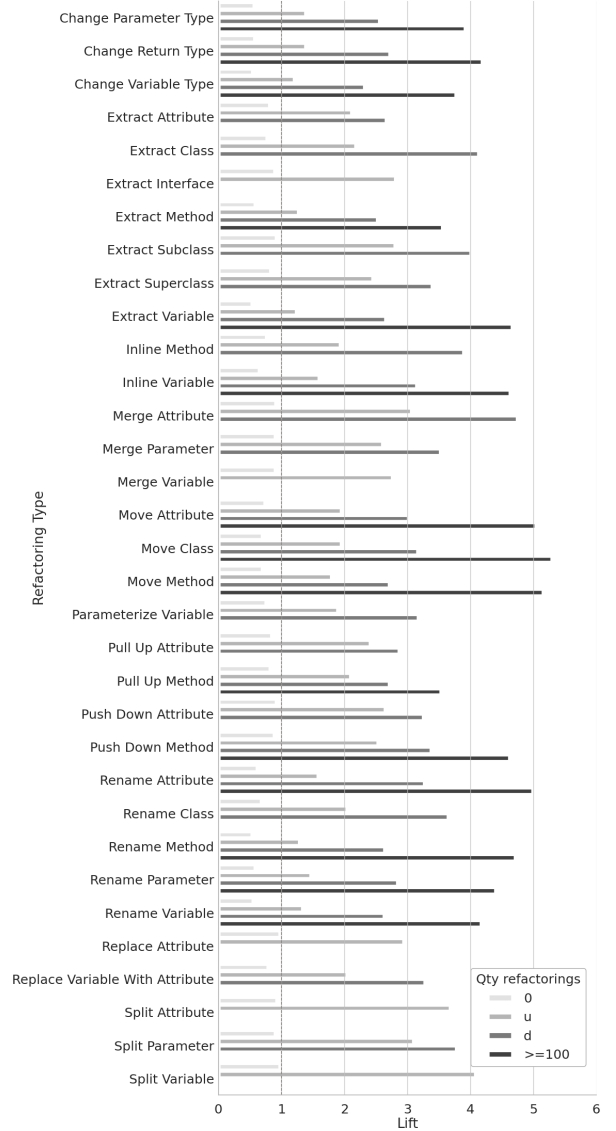


Fig. 3. Influence of each refactoring type on the occurrence of merge effort ($effort = \text{“true”}$)

focused on analyzing which refactorings tend to result in non-zero merge effort.

Our previous work [23] shows that the number of refactorings is associated with merge effort. Figure 3 confirms that this pattern persists for individual refactoring types. For all types, refactoring absence (“0” – lighter gray bar) shows a lower frequency of merge effort occurrences ($Lift < 1$). Intuitively, *Lift* values greater than 1 indicate that merge effort occurs more frequently than expected given the baseline distribution, while values lower than 1 indicate a lower-than-expected occurrence. Thus, a *Lift* of 3.00 means that the merge effort occurs approximately three times more frequently in the presence of the antecedent than expected under independence.

For instance, when the *Extract Method* refactoring is absent in both merge branches ($Extract\ Method = \text{“0”} \rightarrow effort = \text{“true”}$) yields a *Lift* of 0.59, meaning that merge effort occurs 41% less frequently. In contrast, merges that involve the *Extract Method* refactoring show higher frequencies of

merge effort, even in small quantities: “u” – 28% increase ($Lift = 1.28$), “d” – 154% increase ($Lift = 2.54$), and “ ≥ 100 ” – 256% increase ($Lift = 3.56$). All corresponding graph bars show $Lift$ values greater than 1. In nearly all scenarios, except *Extract Attribute*, $Lift$ consistently increases as the number of implemented refactoring increases. For instance, implementing a large number of *Rename Attribute* refactorings (“ ≥ 100 ”) resulted in a $Lift$ apex of 5.00, indicating that merge effort occurs 400% more frequently.

For certain types of refactoring, the graph reveals that we did not identify rules that meet the minimum support threshold (20 merge commits) for all discretization ranges. The absence of bars for types such as *Split* refactorings (the last three on the x-axis), which separate attributes, parameters, and variables, reflects their low overall frequency. This outcome arises as some refactoring types are inherently less frequent. A significant $Lift$ increase occurs for the three *Split* types when increasing from zero (“0”) to units (“u”). Results show that implementing even a few refactorings of these rarer types can significantly increase the chances of having a merge effort. Notably, among the three *Split* refactorings, *Split Parameter* appears more frequently, also meeting the minimum support threshold for the “d” range, where it exhibits a $Lift$ of 3.79 (+279%), following the same increasing trend observed across discretization levels.

Building on this observation, several refactoring types exhibit a clear increasing pattern in the occurrence of merge effort as their frequency increases, forming a “staircase” effect across the discretization bands. Among the most critical cases, *Move Class*, *Move Method*, and *Move Attribute* stand out, all reaching $Lift$ values above 5.00 in the “ ≥ 100 ” range. For instance, *Move Class* increases from 1.96 in “u” to 5.31 in “ ≥ 100 ”, while *Move Method* rises from 1.80 to 5.16, and *Move Attribute* from 1.96 to 5.02. Other refactorings also demonstrate strong growth patterns, such as *Extract Variable* (from 1.25 to 4.67), *Inline Variable* (from 1.61 to 4.64), and *Push Down Method* (from 2.54 to 4.63). In addition, type-changing refactorings such as *Change Return Type* and *Change Parameter Type* show consistent increases, reaching 4.20 (+320%) and 3.93 (+293%), respectively. Renaming operations also follow this pattern, with *Rename Attribute* showing one of the steepest escalations (from 1.59 to 5.00), and other renaming refactorings such as *Rename Method*, *Rename Parameter*, and *Rename Variable* presenting increases above 200% when comparing “u” to “ ≥ 100 ”. Overall, these results indicate that certain refactoring types not only correlate with merge effort, but that their impact intensifies substantially as their volume increases, highlighting a compounding effect of structural changes during parallel development.

To provide a more fine-grained analysis of merge effort, we extended its representation beyond a binary formulation to four levels: *none*, *low*, *medium*, and *high*. To mitigate the impact of the highly imbalanced distribution, we calibrated the discretization thresholds to obtain comparable support values across non-zero categories, focusing on the support of the consequent, as $Lift$ is influenced by its marginal probability. Specifically, we partitioned the range of effort > 0 into three equal-frequency levels: *low* (> 0 and ≤ 5), *medium* (> 5 and

≤ 20), and *high* (> 20), each representing approximately one third ($\approx 33.33\%$) of the non-zero instances. This configuration produced similar support values (*low*: 0.0222, *medium*: 0.0237, *high*: 0.0200), reducing the influence of consequent support on $Lift$ and enabling more balanced comparisons. We then selected the five refactoring types most strongly associated with refactoring quantity and merge effort for analysis: *Change Return Type*, *Extract Variable*, *Move Class*, *Rename Attribute*, and *Split Parameter*.

Focusing on *Change Return Type*, we observed a clear and consistent progression across both refactoring quantity and effort levels. For *low* effort, the $Lift$ increases from 0.73 to 1.33, 2.02, and 2.16 across “0”, “u”, “d”, and “ ≥ 100 ”, respectively, corresponding to incremental increases of +82.19% (“0” to “u”), +51.88% (“u” to “d”), and +6.93% (“d” to “ ≥ 100 ”). For *medium* effort, the progression becomes more pronounced, with $Lift$ values rising from 0.56 to 1.52, 2.62, and 3.49, resulting in increases of +171.43%, +72.37%, and +33.21%, respectively. The strongest escalation appears for *high* effort, where $Lift$ increases from 0.44 to 1.30, 3.66, and 7.31, corresponding to +195.45%, +181.54%, and +99.73%. These results indicate that, although the absence of this refactoring shows a negative association with merge effort ($Lift < 1$), increasing quantities strongly shift the association toward higher effort levels, particularly for *high* effort.

The remaining refactoring types exhibit similar progression patterns across effort levels, although they vary in magnitude and, in some cases, lack sufficient support for higher ranges. For *Extract Variable*, the incremental increases from “0” to “u”, “u” to “d” and “d” to “ ≥ 100 ” are +98.46%, +55.04%, and +14.00% for *low* effort; +151.92%, +97.71%, and +65.25% for *medium*; and +153.33%, +205.26%, and +124.14% for *high*, indicating a consistent upward trend across all effort levels.

For *Move Class*, we observed increases of +114.81% and +28.16% for *low* effort (“0” to “u” and “u” to “d”), although no rule meets the minimum support threshold for ≥ 100 in this level; for *medium*, the progression continues with +200.00%, +21.60% and +84.17%; and for *high*, with +238.98%, +146.50%, and +103.65%. A similar pattern appears for *Rename Attribute*, with increases of +105.41% and +41.45% for *low* effort (with no support for ≥ 100); +170.49%, +90.91%, and +33.97% for *medium*; and +220.00%, +192.50%, and +105.56% for *high*. Finally, *Split Parameter*, despite its lower frequency, follows the same overall trend whenever sufficient support is available, with increases of +130.53% for *low*, +192.47% for *medium*, and +445.24% for *high* in the transition from “0” to “u”; however, subsequent ranges (“u” to “d” and “d” to “ ≥ 100 ”) do not meet the minimum support threshold. Overall, these results indicate that the incremental growth of association strength remains consistent across different refactoring types, even in the presence of data sparsity for less frequent transformations.

To illustrate in practice the merge effort associated with certain types of refactorings, we conducted a manual inspection of the combined diff of a merge commit (`bcaflc`) from the *shardingSphere* project, focusing on the *Move Class* refactoring type. In our results, *Move Class* showed a strong

association with the occurrence of merge effort, as indicated by increasing *Lift* values with higher numbers of such refactorings across the merge branches. In this commit, RefactoringMiner detected 272 *Move Class* refactorings, while the merge effort amounted to 31 lines added or removed. From the combined diff, we identified 19 added import statements, a subset of which is illustrated in Code 2. Manual inspection of the source code confirmed that these changes correspond to 14 distinct classes that were moved from the same package as their callers to different packages, requiring explicit import declarations. This provides concrete evidence that structural refactorings, such as *Move Class*, directly contribute to merge effort through dependency adjustments.

Code 2 Example of merge effort associated with *Move Class* refactorings in commit `bcaflc` of the *shardingSphere* repository.

```
++ import org.apache(...).SQLStatement;
++ import org.apache(...).expression.SqlExpression;
```

Answer to RQ1: Refactoring types exhibit a “staircase” pattern: higher frequencies consistently correlate with increased merge effort. While the absence of refactorings shows low association (*Lift* < 1), increasing volumes yield progressively higher *Lift* values across *low*, *medium*, and *high* effort levels. Structural refactorings show the strongest associations: *Move Class* reached *Lift* 5.31 (+431%), followed by *Move Method* and *Move Attribute*. Renaming operations also escalate steeply, notably *Rename Attribute* (*Lift* 5.00; +400%). Local transformations, such as *Extract Variable* (*Lift* 4.67; +367%) and *Inline Variable* (*Lift* 4.64; +364%), demonstrate similar growth. Type-changes (Eg. *Change Return Type*) reached *Lift* 4.20 (+320%), while even infrequent operations like *Split Parameter* showed substantial impact (*Lift* 3.79; +279%). These results suggest a compounding effect during parallel development, in which both refactoring type and volume can intensify the occurrence and magnitude of merge effort.

B. RQ2: Does the amount of different refactoring types influence the merge effort?

This research question examines whether the variety of refactoring types is associated with a higher frequency of merge effort, regardless of the total number of refactorings. Because the number and diversity of refactorings often correlate, it is essential to disentangle their individual effects to determine whether diversity offers explanatory power beyond refactoring volume. We first explored the relationship between the total number of refactorings (TNR), the number of different refactoring types (NDRT), and merge effort (discretized as *true* or *false*), using boxplots shown in Figure 4. The plot shows that both attributes present higher values when the effort is *true*. Specifically, the median TNR rises from 4 to 46, and the median NDRT increases from 3 to 10. These raw changes correspond to 1,050% and 233% increases, respectively. However, due to the differing scales of the attributes (TNR: 0 – 418; NDRT: 0 – 33), proportional shifts relative to range provide more balanced insight – approximately 10% for TNR and 21% for NDRT. This normalization suggests that NDRT shows a relatively more pronounced shift when adjusted for scale, despite TNR’s larger absolute growth.

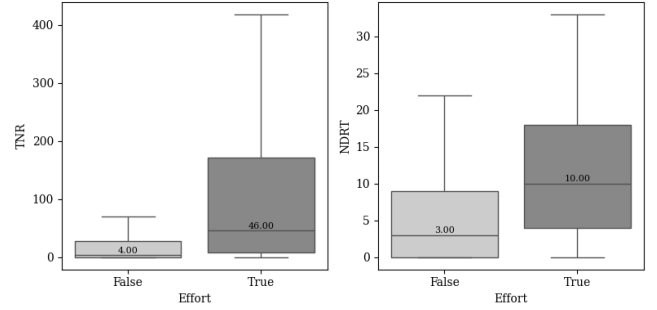


Fig. 4. Distribution of the TNR and NDRT attributes considering the occurrence of merge effort (*true* or *false*).

Since we observed that $TNR > 0$ consistently implied $NDRT > 0$, we hypothesized a strong association between these variables. To test this, we computed Spearman’s [52] rank correlation, which confirmed a strong positive monotonic relationship ($\rho = 0.9771$, $p < 0.0001$). However, we also needed to verify whether these attributes provided non-redundant information. For this purpose, we calculated the Variance Inflation Factor (VIF) [53], which measures the extent of multicollinearity among predictors. The resulting VIF of 1.31, well below the commonly accepted threshold of 5 [53], indicates low collinearity, supporting the interpretation that TNR and NDRT can be considered independently in subsequent analyses.

Given the strong correlation between TNR and NDRT, we applied the Apriori algorithm [49] to analyze whether each attribute, individually or in combination, is associated with the occurrence of merge effort. We collected association rules where the antecedent combines TNR and NDRT values, and the consequent represents the occurrence of merge effort. To visualize results, we constructed a heatmap (Figure 5). Each cell shows the *Lift* value of a rule satisfying the 20-tuple minimum support threshold. Specifically, we collected rules in the format: $TNR = “i” \wedge NDRT = “j” \rightarrow effort = “true”$, where i represents TNR (row) and j NDRT (column). Empty cells indicate combinations yielding no rules due to failing the minimum support threshold. A practical example occurs when $TNR = 93$ and $NDRT = 18$ (rule: $TNR = 93 \wedge NDRT = 18 \rightarrow effort = “true”$), resulting in a *Lift* of 3.79 (+279%).

To assess the individual influence of TNR and NDRT on merge effort, we analyzed percentage variations based on heatmap *Lift* values. For each fixed TNR (rows), we performed adjacent pairwise comparisons by varying NDRT (columns). Then, we repeated this by fixing NDRT (columns) and varying TNR (rows). For each pair ($c1, c2$), where $c1$ is the preceding cell and $c2$ the subsequent one within the same row or column (both containing the *Lift* value), we calculated the percentage difference using the formula: $(Lift_{c2} - Lift_{c1}) / Lift_{c1}$. This metric quantifies the relative increase or decrease between consecutive *Lift* values. For instance, fixing $NDRT = 2$ and varying TNR yields four pairwise percentage differences: between 1.07 and 0.82 = $(1.07 - 0.82)/0.82 = 30.49\%$; between 0.82 and 0.54 = $(0.82 - 0.54)/0.54 = 51.85\%$; between 0.54 and 0.62 = $(0.54 - 0.62)/0.62 = -12.90\%$; and between 0.62 and 0.46 = $(0.62 - 0.46)/0.46 = 34.78\%$.

We excluded comparisons involving isolated values or gaps

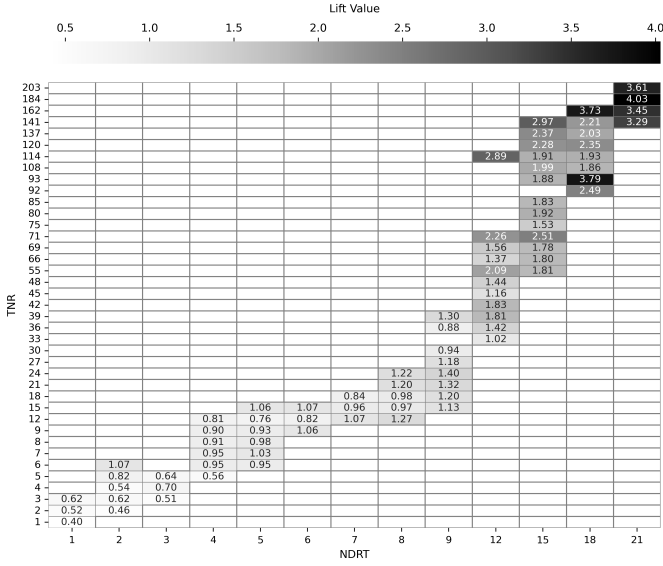


Fig. 5. Heatmap showing the *Lift* values of the extracted association rules: $TNR = "value_i"$ and $NDRT = "value_j" \rightarrow effort = "true"$, where i represents the row and j the column.

between valid entries to avoid misleading variations. For example, fixing $TNR = 6$ and varying $NDRT$, we excluded 1.07 ($TNR = 6$ and $NDRT = 2$) due to a gap before 0.95 ($NDRT = 4$). Similarly, we excluded 2.89 in column-wise comparisons when a gap existed after 2.26 ($TNR = 71$ and $NDRT = 12$). After filtering, we computed the mean and standard deviation of the resulting percentage differences. TNR comparisons yielded a mean of 9.28% and a standard deviation of 0.259. $NDRT$ -related comparisons showed a mean of 7.88% and a standard deviation of 0.258. Although modest, the results suggest a slightly stronger association for TNR , while indicating that both attributes contribute to the merge effort.

To complement the previous analysis and address merge effort intensity, we analyzed association rules with consequents representing effort levels (*low*, *medium*, *high*), using the same discretization as in RQ1. We focused on representative $NDRT$ groups (*little*, *some*, and *much*), ensuring minimum support across all effort levels. For the *little* diversity group: when $NDRT = 2$, all associations present *Lift* values below 1: 0.69 (-31%), 0.59 (-41%), 0.40 (-60%); indicating consistently lower frequencies of merge effort. A similar but weaker pattern is observed for $NDRT = 4$: 0.99 (-1%), 0.97 (-3%), 0.65 (-35%); where *low* and *medium* remain near independence, but *high* effort is still reduced ($\approx 35\%$, $Lift=0.65$). As diversity increases, associations become consistently positive. For *some* group: when $NDRT = 12$, *Lift* values are 1.55 (+55%), 1.82 (+82%), and 1.36 (+36%), increasing to 1.61 (+61%), 2.15 (+115%), and 1.69 (+69%) for $NDRT = 14$. This trend intensifies at higher levels in the *much* diversity group: $NDRT = 22$ yielding 2.29 (+129%), 3.20 (+220%), and 3.99 (+299%), and $NDRT = 26$ reaching 3.00 (+200%), 2.51 (+151%), and 5.81 (+481%) for *low*, *medium*, and *high*, respectively. This pattern becomes more pronounced at high levels of diversity. For instance, $NDRT = 28, 29$, and 31 are strongly associated with *high* effort, with *Lift* values of 6.67 (+567%), 6.18

(+518%), and 14.16 (+1,316%), respectively. These cases indicate that merges involving many distinct refactoring types are substantially more likely to incur high merge effort.

Overall, the results show that not only the volume but also the diversity of refactorings contribute to effort severity. While lower $NDRT$ values are associated with reduced or near-independent effort levels, higher diversity consistently shifts the association toward more complex and effort-intensive merge scenarios, highlighting structural heterogeneity as a relevant risk factor.

Answer to RQ2: The number of different refactoring types ($NDRT$) and the total number of refactorings (TNR) both show significant and independent associations with merge effort. On average, each increase in $NDRT$ corresponds to a 7.88% rise in the frequency of merge effort occurrences, even when TNR remains constant. Similarly, a 9.28% increase in refactoring volume (TNR) results in a proportional rise in merge effort frequency. These results highlight the independent and relevant contribution of diversity to the occurrence of merge effort and emphasize the importance of jointly considering both dimensions when evaluating or planning parallel code changes. Furthermore, this pattern is consistently observed across different levels of merge effort (*low*, *medium*, and *high*), with higher refactoring diversity associated with progressively stronger associations with higher effort levels.

C. RQ3: How do combinations of refactoring types, applied concurrently in different branches, influence the merge effort?

This RQ investigates whether specific combinations of refactoring types, applied concurrently in parallel branches, are associated with merge effort. Figure 6 presents a heatmap in which each cell reports the *Lift* value of a refactoring pair under the condition $effort = true$. We consider refactoring co-occurrence across branches irrespective of which branch each transformation was applied to. As described in Section II-C, *Lift* values were recalculated under a symmetric formulation in which refactoring pairs are considered independently of the branch in which they occur. For instance, the rules *Extract Method* in branch 1 and *Extract Interface* in branch 2 $\Rightarrow effort = "true"$ and its inverse were treated as a single combined rule. This aggregation captures refactoring co-occurrence across branches, regardless of ordering, thereby requiring recomputation of *Lift*.

We analyzed 520 refactoring pairs that satisfied the minimum support threshold of 20 tuples. Most combinations exhibited *Lift* values above 3, with an average of 5.17 (range 3.05–9.48), indicating that concurrent refactorings across branches are often associated with a higher frequency of merge effort. Notably, 54.0% of the pairs presented *Lift* values greater than 5.0, suggesting that this effect is widespread rather than limited to a small subset of combinations.

To obtain a comprehensive understanding of how refactoring combinations are associated with merge effort, we adopted a multidimensional analysis that integrates association strength (*Lift*), effect size (Odds Ratio), statistical significance (Fisher's Exact Test), and support of the rule. All reported p-values are below 0.0001. Given the large size of our dataset (91,270 tuples) and the imbalanced distribution of the effort variable (approximately 93% of cases have $effort = "false"$), statistical

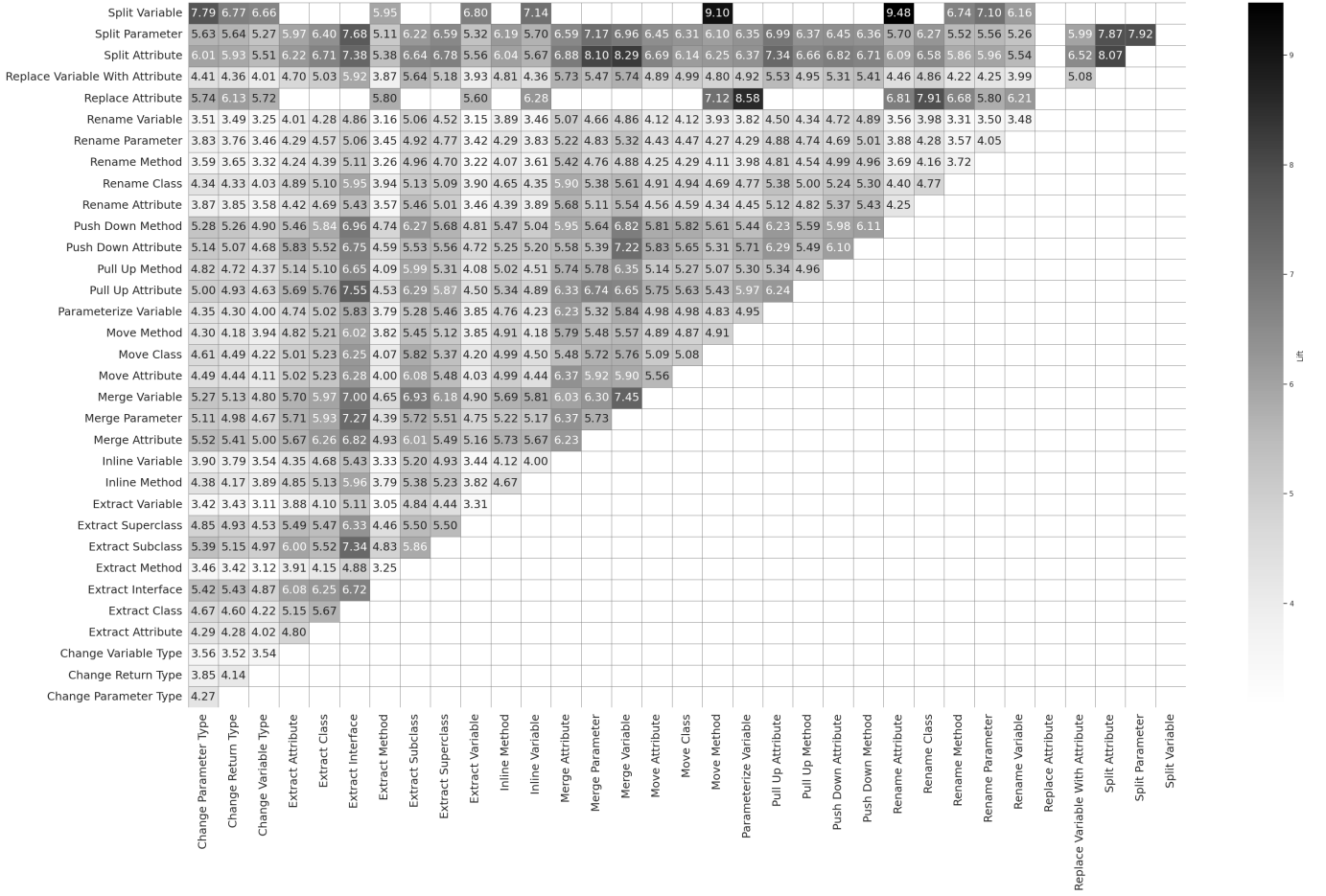


Fig. 6. Heatmap of *Lift* values associating refactorings introduced in parallel branches under the condition *effort* = *true*.

significance alone is insufficient to assess practical relevance. Therefore, we focused on the effect size (Odds Ratio) and the support of the rule to assess practical relevance. Based on these dimensions, we classify refactoring pairs into two categories according to their effect magnitude and robustness.

Rare but high-impact combinations correspond to pairs with very high effect magnitude but low support (Odds Ratio ≥ 10 , high *Lift*, and support ≥ 20 and ≤ 50). In this group, the Odds Ratio (OR) threshold captures extreme effect sizes, indicating that the frequency of merge effort is at least ten times higher when these refactoring pairs occur compared to when they do not. At the same time, the support range indicates that these combinations are relatively infrequent in the dataset, meaning that their impact is concentrated in a limited number of merge scenarios. Although not among the most frequent patterns, these combinations exhibit stable and extreme associations with merge effort. In total, we identified 10 refactoring pairs that satisfy these criteria.

Representative examples include *Inline Variable* with *Split Variable* (*Lift* = 7.14, OR = 12.69, support = 49), *Merge Attribute* with *Split Attribute* (*Lift* = 6.88, OR = 11.84, support = 45), *Push Down Attribute* with *Split Attribute* (*Lift* = 6.82, OR = 11.65, support = 37), and *Extract Subclass* with *Split Attribute* (*Lift* = 6.64, OR = 11.52, support = 29).

These combinations often involve structural transformations that fragment, merge, or reorganize shared data structures, creating semantically incompatible changes across branches. In particular, many of these patterns involve operations that alter data representation or redistribute responsibilities across program elements, which can increase the chances of inconsistencies when applied concurrently in parallel branches.

Very frequent and high-impact combinations correspond to pairs that jointly exhibit high effect magnitude and very high support (Odds Ratio ≥ 7 and support ≥ 500), typically accompanied by consistently high *Lift* values. In this group, the Odds Ratio threshold identifies combinations with strong, stable effects, while the support constraint captures highly recurrent patterns in the dataset. Unlike rare high-impact combinations, these patterns occur frequently in our dataset, making them particularly relevant in practice.

Across this group, we identified 28 refactoring pairs involving 16 distinct refactoring types out of the 33 considered in this study. These types can be grouped into three main categories: (i) structural transformations (e.g., *Extract Class*, *Move Method* and *Pull Up Attribute*), which modify program organization and class hierarchies; (ii) interface and contract changes (e.g., *Rename Method*, *Change Parameter Type* and *Parameterize Variable*), which affect method signatures and

Code 3 Example of merge effort identified in commit e82e72d of the *OpenRefine* repository.

```
- public void setRowCount(long totalCount) {
+ public void setRowCount(long rowCount) {
- public void setRowCount(long totalCount) {
+ public void setRowCount(int totalCount) {
++ public void setRowCount(long rowCount) {
```

data-structure representations; and (iii) local implementation adjustments (e.g., *Inline Method* and *Replace Variable With Attribute*), which operate at a finer granularity. Representative examples include *Extract Class* with *Extract Class* (*Lift* = 5.67, *OR* = 8.79, *support* = 521), *Move Attribute* with *Move Class* (*Lift* = 5.09, *OR* = 7.70, *support* = 508), *Extract Superclass* with *Rename Attribute* (*Lift* = 5.01, *OR* = 7.55, *support* = 538), and *Move Class* with *Rename Class* (*Lift* = 4.94, *OR* = 7.48, *support* = 606).

Notably, the most frequent and high-impact combinations are predominantly formed by interactions between structural transformations and interface-level changes. This suggests that merge effort is associated with concurrent modifications that alter program structure and exposed interfaces. Such combinations tend to produce semantically related but structurally incompatible changes across branches, which may make automatic resolution more challenging. This result suggests that a relatively small subset of refactoring types accounts for a large share of the most impactful and recurrent merge scenarios.

Overall, these results indicate that merge effort is associated with the interaction of concurrent structural changes across branches, rather than isolated refactoring operations. This pattern holds for both rare but high-impact and very frequent combinations, suggesting that similar interaction mechanisms occur across both extreme and recurrent scenarios. The most critical cases arise when transformations modify shared data representations, method signatures, or class hierarchies, producing semantically related but structurally incompatible changes that are difficult to reconcile automatically. However, the presence of a refactoring pair across parallel branches does not necessarily imply that both refactoring types contributed to the same regions of code in conflict. Refactoring pairs are not isolated within merge scenarios, as multiple other refactoring types and code changes often coexist in the same branches.

To illustrate how the interaction of refactorings across parallel branches can lead to merge effort, we present a concrete example of a textual conflict involving two distinct refactoring types. Code 3 shows a merge scenario identified in commit e82e72d from the *OpenRefine* project. In this example, two branches introduced different refactorings to the same method. One branch applied a *Rename Parameter*, changing the parameter name from *totalCount* to *rowCount*, while the other applied a *Change Parameter Type*, modifying its type from *long* to *int*. During the merge, the developer manually reconciled these conflicting transformations by preserving the renamed parameter introduced in one branch while keeping the original *long* type, thus rejecting the change to *int*. This example highlights how even simple, common refactorings, when applied concurrently, can lead to merge effort that is not fully resolved automatically by merge tools.

Answer to RQ3: Merge effort is associated with the co-occurrence of refactoring types across parallel branches, rather than with isolated transformations. We observed two complementary patterns: rare but high-impact combinations, typically involving structural transformations that fragment or redistribute program elements (e.g., *Split Variable*, *Split Attribute*), often combined with operations that reorganize or consolidate structure (e.g., *Merge Attribute*); and very frequent and high-impact combinations, characterized by interactions between structural reorganization (e.g., *Extract Class*, *Move Method*) and changes to method signatures and data-structure representations (e.g., *Rename Method*, *Change Parameter Type*, *Parameterize Variable*). Overall, these results indicate that merge effort is primarily associated with concurrent refactorings, although this relationship should be interpreted as associative rather than causal.

D. Discussion and Implications

This section interprets the results and presents practical implications.

Our results can be explained through concrete code-level merge scenarios. When refactorings occur in parallel and affect the same code region, structural transformations such as *Move Class*, *Move Method*, and *Move Attribute* often induce textual conflicts if one branch relocates code while another modifies its original location, leading to overlapping deletions and edits. Similarly, *Rename Attribute* and *Change Return Type* may conflict with parallel edits that still reference previous identifiers or types, producing inconsistent method signatures or field usages.

Even localized transformations, such as *Extract Variable*, *Inline Variable*, and *Split Parameter*, can introduce conflicts when one branch rewrites expressions while the other modifies them. Such conflicts do not require refactorings in both branches, but often arise from interactions between refactorings and regular edits. When refactorings are applied in both branches, combinations such as *Move Attribute* + *Move Class* may produce incompatible structural reorganizations, while combinations like *Extract Superclass* + *Rename Attribute* and *Move Class* + *Rename Class* introduce inconsistencies in class hierarchies and naming, generating conflicts in shared structures (e.g., class definitions and method signatures) that cannot be automatically merged.

However, not all merge effort is explained by textual conflicts. In some cases, refactorings applied in parallel branches can lead to semantically inconsistent code that compiles or merges without explicit conflicts but requires manual adjustments during integration (e.g., adapting method calls, updating dependencies, or reconciling partially moved logic). These situations result in additional lines added or removed in the merge commit, reflecting the effort required to restore functional correctness rather than resolve direct textual conflicts.

To contextualize our refactoring-based findings, we examined two baseline indicators of code changes: *number_lines_changed* and *number_files_changed* across the merging branches. We analyzed these attributes using the same association rule mining procedure described in Section II-C, based on the Apriori algorithm and the *Lift* measure, with the same minimum support threshold of 20 merge commits.

To enable rule extraction, both metrics were discretized into quartile-based ranges: low (>0 and $\leq Q1$), medium

(> $Q1$ and $\leq$ median), high (> median and $\leq Q3$), and very high (> $Q3$). In our dataset, these correspond to: for *number_lines_changed*, low (>0 and ≤ 175), medium (>175 and ≤ 975), high (>975 and $\leq 5,817$), and very high (>5,817); and for *number_files_changed*, low (>0 and ≤ 9), medium (>9 and ≤ 30), high (>30 and ≤ 125), and very high (>125).

When merge effort is analyzed as a binary attribute (*effort* = “true”), both metrics exhibit a consistent pattern: *low* values are associated with substantially lower frequencies of merge effort, while *very high* values are associated with substantially higher frequencies. For example, *number_lines_changed* yields *Lift* values ranging from 0.16 (-84%) in the *low* range to 2.50 (+150%) in the *very high* range, while *number_files_changed* ranges from 0.18 (-82%) to 2.45 (+145%).

We then analyzed effort intensity using the same discretization adopted in RQ1 (*low*, *medium*, and *high*). Under this setting, *very high* values of *number_lines_changed* yield *Lift* values of 2.22 (+122%), 2.64 (+164%), and 2.98 (+198%) for *low*, *medium*, and *high* effort, respectively, while *number_files_changed* yields corresponding values of 2.21 (+121%), 2.56 (+156%), and 2.85 (+185%). These results show that larger change volumes are associated with both the occurrence of merge effort and higher effort intensity.

These findings confirm basic change volume metrics capture relevant aspects of merge complexity. In particular, higher numbers of changed lines and files are consistently associated with higher merge effort frequencies and intensities. However, when contrasted with our refactoring analysis, an important distinction emerges. While churn-related metrics reach *Lift* values around 2.5–3.0, several refactoring types and combinations exhibit substantially stronger associations. For example, major *Rename Attribute* refactorings reach *Lift* values above 5.0, and certain concurrent refactoring combinations exceed *Lift* values of 8.0. These results imply that refactoring-aware signals capture aspects of merge complexity that goes beyond the sheer volume of changes, providing a fine-grained perspective on how code transformations relate to merge effort — a motivational insight for the practical guidelines discussed next.

Practical Guidelines. Our findings suggest that refactorings, particularly structural ones and their concurrent application across branches, are often associated with increased merge effort. These results highlight the importance of coordination strategies, such as reducing branch divergence, integrating changes more frequently, and improving communication around refactoring activities. From a tooling perspective, they also indicate opportunities for refactoring-aware merge strategies and recommendation systems that consider refactoring type, volume, diversity, and cross-branch combinations.

To operationalize these implications, Table III summarizes a set of actionable guidelines derived from RQ1–RQ3. The table organizes recurring refactoring scenarios, links them to recommended actions, and illustrates how to implement these recommendations in practice through pull request analysis, continuous integration pipelines, and repository-level tooling (e.g., GitHub Actions, Gerrit hooks, and bots) to support more informed merge decisions and reduce integration complexity. While each guideline is associated with a specific observed scenario, many recommendations are inherently cross-cutting

and applicable to multiple contexts. We organize them according to their most representative scenarios to maintain clarity and alignment with our empirical findings.

IV. THREATS TO VALIDITY

This section discusses internal, external, construct, and conclusion threats that may have influenced the reported results.

Internal Validity. One potential threat concerns the completeness of the collected data on refactorings. We imposed a 10-minute *RefactoringMiner* timeout, following literature recommendations [20] to prevent analysis from hanging on long-running commits. In rare cases (only 94 commits, representing 0.0001% of our dataset), the process terminated after exceeding this threshold. Although this is a negligible portion of the data, non-detected refactorings may pose a minor threat to the validity of aggregated results.

We analyzed co-occurrence patterns using association rules, interpreting them as dataset-specific associations rather than evidence of causality. This technique identifies frequent combinations based on support, confidence, and lift, enabling the detection of refactoring patterns associated with higher code churn during merges. However, it does not control for latent factors such as project size, branching policies, or system modularity, and relies on code churn as a proxy for merge effort, limiting causal inference and internal validity. To mitigate spurious conclusions, we conducted sensitivity analyses and stratified results by the total number of refactorings (TNR) and the number of different refactoring types (NDRT). Nonetheless, future studies could further investigate causal mechanisms and assess the generalizability of these findings in other contexts.

Discretizing numeric attributes into binary or categorical ranges, required by the Apriori algorithm, introduces an internal validity threat by reducing data granularity and creating arbitrary thresholds. Such discretization may homogenize distinct behaviors (e.g., 10 vs. 90 refactorings) and obscure subtle patterns. We mitigated this by using two strategies: a four-tier discretization (0, 1–9, 10–99, ≥ 100) that aligns with natural data clusters and highlights rare, high-impact cases (RQ1), and a binary form (0 vs. >0) to simplify presence analysis and control rule volume (RQ2 and RQ3). While some precision is lost, the trade-off enabled interpretable pattern discovery at scale, and the study still revealed a substantial number of consistent associations that support the conclusions discussed in Section III-D. Future work can explore dynamic discretization techniques [56], [57], [58] to capture finer-grained patterns.

Although our merge effort metric does not focus exclusively on textual conflicts, developers may only detect build, test, or production conflicts after completing the merge. Ghiotto et al. [1] refer to these situations as postponed merges, in which developers resolve conflicts through changes committed after the merge commit. Nevertheless, their study found that such cases are rare, representing about 0.97% of the analyzed merge

¹<https://www.gerritcodereview.com/>

²<https://slack.com/>

³<https://discord.com/>

TABLE III
ACTIONABLE GUIDELINES AND TOOLING SUPPORT DERIVED FROM REFACTORING-MERGE EFFORT ASSOCIATIONS

Observed Scenario	Practical Guideline	Tooling/Workflow Implementation (e.g., GitHub, Gerrit, CI)
High volume of refactorings (High TNR)	Consider splitting the refactoring into smaller increments or integrating changes earlier to reduce accumulation.	CI/CD Gate with Automated Bot Warnings: Implement a GitHub Action with a rule-based bot that detects Pull Requests exceeding a predefined TNR threshold, automatically issuing warnings and suggesting earlier merges. The bot can also enforce mandatory senior reviews, acting as a CI/CD gate to mitigate risks associated with high volumes of refactorings.
High diversity of refactoring types (High NDRT)	Monitor integration risk as heterogeneous transformations increase overlapping edits.	Integration Assistant: Deploy a bot to provide PR-level analytics on refactoring diversity, displaying risk indicators in GitHub checks or Gerrit dashboards ¹ to alert reviewers about complex structural changes and support more informed review decisions.
Structural refactorings (e.g., Move Class, Push Down Method)	Avoid concurrent applications across branches. Prioritize developer communication if unavoidable, and consider isolating large structural refactorings into dedicated branches when possible.	Structural Refactoring Conflict Predictor: Use a Gerrit hook or GitHub bot integrated into CI pipelines to detect structural refactorings (e.g., via RefactoringMiner) and identify concurrent changes across Pull Requests that modify the same class hierarchy. The system can proactively alert developers about potential conflicts arising from overlapping structural changes at the repository level.
Concurrent refactorings on related code elements	Review Pull Requests for interactions between refactorings and parallel edits. Sequence changes to minimize interference, and promote early communication among developers when such changes are unavoidable.	Dependency Mapping with Cross-Branch Analysis: Integrate refactoring detection techniques into CI pipelines to visualize impacted code regions directly in the Pull Request dashboard, while performing cross-branch analysis to detect overlapping edits or refactorings across active branches and proactively notify developers before integration (e.g., using tools such as RefactoringMiner).
Presence of high-risk combinations (RQ3 Results)	Increase attention during code review and merge preparation for known high-risk pairs (e.g., Split vs. Extract refactorings), with tool support that automatically flags these combinations and assists reviewers.	High-Risk Refactoring Linter/Bot with Escalation: Deploy a rule-based bot (e.g., GitHub Actions) that detects high-Lift refactoring combinations, automatically labels Pull Requests as “High Integration Risk”, and suggests safer structural alternatives. Additionally, the bot can trigger automated alerts in team communication channels (e.g., Slack ² and Discord ³) to support immediate coordination and risk-aware integration decisions.
Refactoring-heavy branches vs. feature work	i) Establish dedicated branches to isolate massive structural modifications. ii) Prioritize the integration of refactoring-heavy branches, prefer short-lived branches, and reduce prolonged parallel development on affected modules. iii) Consider adopting trunk-based development with feature toggles [54], [55] to reduce prolonged parallel divergence by enabling early integration and limiting the accumulation of changes.	Refactoring-Aware Merge Control and Coordination: Use Gerrit’s voting or GitHub’s “Draft Pull Request” status to temporarily block feature merges until structural refactoring is integrated, while leveraging dashboards that indicate branch divergence and refactoring density to support merge prioritization and coordination decisions.
Refactoring-aware merge strategies and timing	Consider merge strategies that separate edits from refactorings and schedule merges based on refactoring thresholds to reduce integration complexity.	Operation-Based Merge and Timing Assistant: Implement merge workflows that first integrate non-refactoring edits and then replay refactorings (inspired by operation-based merging [5]). Additionally, use CI-integrated tools to monitor refactoring counts across branches and recommend optimal merge timing, triggering alerts when thresholds indicate increased merge effort risk.

scenarios, suggesting that postponed resolutions have a limited impact on our overall results.

External Validity. Our study focuses on open-source Java projects, and we do not claim that our findings generalize to all Java systems, particularly to closed-source enterprise software or to projects written in other programming languages. Selecting mature, well-established repositories may limit applicability to less mature projects that may exhibit different development dynamics. Additionally, restricting the dataset to primarily English-language repositories and excluding projects with extremely high or low commit counts may have reduced diversity, potentially underrepresenting certain refactoring and merge scenarios. Therefore, the generalizability of our findings is limited to the characteristics of the selected sample. Despite these constraints, our analysis revealed consistent behavioral patterns, suggesting that similar trends may exist in other contexts and warrant further investigation.

Construct Validity. To operationalize merge effort, we relied on code churn, measured as the number of lines added or removed during merge commits. This metric has been widely adopted in software engineering as a proxy for effort and change complexity [22], [41], [42], [43], [44]. However, code churn captures only one observable dimension of merge effort, primarily reflecting the magnitude of code modifications required during integration, and may not fully represent other

important aspects, such as cognitive load, coordination among developers, and the time required to resolve conflicts. As a result, scenarios involving large but straightforward changes (e.g., rename-based refactorings supported by automated tools) may be overestimated in terms of effort, while small but complex conflicts requiring careful reasoning may be underestimated. To mitigate this limitation, we explicitly frame code churn as a proxy for merge effort and interpret our results within this scope, while complementing our analysis with different perspectives on change (e.g., refactoring types, diversity, and combinations), which provide a more fine-grained view beyond change volume alone.

In this study, refactorings are identified in the branches’ development history prior to merge commit. Thus, our analysis focuses on refactoring activities introduced before the integration step and does not include refactorings along merge resolution. Although developers may occasionally introduce refactorings while resolving conflicts, this scenario might be generally uncommon, as merge resolution typically focuses on integrating concurrent code changes rather than introducing new structural modifications. Consequently, excluding refactorings introduced during the merge operation may lead to a slight underestimation of refactoring activity in specific cases. This fact does not materially affect our results, as the study investigates associations between refactoring activities made

in parallel branches and merge effort.

Conclusion Validity. We observe that some of the identified association rules show relatively low support, occurring in only 20 merge commits. As a result, some rules may have occurred by chance. However, removing them would eliminate rare yet potentially relevant patterns.

V. RELATED WORK

Several studies explored the relationship between refactorings and merge operations [16], [19], [17], [18], [20], [59], [23]. Most [19], [16], [17], [59] focused on developing, improving, or proposing merge tools that account for refactorings in different branches. Only some studies [20], [23], [60] adopted an analytical perspective, investigating the relationship between refactorings and merge conflicts. This section discusses existing contributions and research gaps.

Angyal et al. [19] extended the three-way merge algorithm to support renames, incorporating them into change reconciliation. Dig et al. [16] introduced *MolhadoRef*, a tool identifying seven refactoring types before code merge using *RefactoringCrawler*. This tool treats refactorings and edits as recorded/replayed change operations. They proposed inverting refactorings, performing a merge, then replaying refactorings to address intermingled edits and refactorings, comparing results against CVS [61]. Leßenich et al. [17] enhanced *JDIME* to resolve conflicts involving specific refactorings like code movements. They proposed a syntax-aware structured merge strategy based on heuristic optimization and syntax-specific lookahead during tree matching.

Ellis et al. [59] further investigated refactoring-aware merging by empirically evaluating operation-based approaches such as *RefMerge* and *IntelliMerge*. Their study analyzed 2,001 merge scenarios extracted from 20 Java open-source projects and evaluated how refactoring-aware algorithms perform compared to traditional merge strategies. Their findings show that refactoring-aware approaches can reduce or automatically resolve a portion of merge conflicts that conventional tools fail to handle. Unlike our work, their study focuses on evaluating merge algorithms rather than analyzing how different refactoring types relate to merge effort. In contrast, our study investigates more than 90,000 merge commits across 64 Java projects and focuses on identifying patterns between refactoring types, their cross-branch combinations, and merge effort using association rule mining.

Mahmoudi and Nadi [18] investigated changes made by phone vendors compared with original Android development, focusing on concurrent evolution and change overlap. They reported common refactorings and examined the feasibility of automated merge support, considering a small subset of six frequent types. Extending this work, Mahmoudi et al. [20] analyzed the relationship between 15 refactoring types and merge conflicts across 3,000 Java open-source projects. They defined “involved refactoring” as occurring in changes that overlap conflicting regions, and classified a merge scenario as involving refactorings when at least one conflicting region contains such a transformation. Their results showed that 22% of merge conflicts involve refactorings, with 11% of conflicting regions including at least one, and that *Extract*

Method appears more frequently in conflicts. Additionally, conflicting regions with refactorings tend to be larger and more complex, and such scenarios often involve more evolutionary changes.

While Mahmoudi et al. [20] provide an analysis of the presence of refactorings in merge conflicts across 3,000 Java projects, their study focuses on identifying whether refactorings are involved in conflicting regions and on characterizing conflict properties, such as their size. In contrast, our study analyzes a curated dataset of 64 projects and investigates a different dimension of the problem by examining how specific refactoring types and their combinations relate to the effort required to resolve merges. From a methodological perspective, Mahmoudi et al. define “involved refactoring” based on overlap with conflicting regions, whereas we adopt an association-rule mining approach that combines *Lift*, Odds Ratios, Fisher’s Exact Test, and support to analyze the relationship between refactoring types and merge effort. While their study considers 15 refactoring types, we extend this analysis with a more fine-grained investigation of 33 types. Mahmoudi et al. report that refactorings are present in a substantial portion of merge conflicts and that such conflicts tend to be larger and more complex. Our results go beyond presence and show that specific refactoring types, their volume, diversity, and cross-branch combinations, are associated with increased merge effort. In particular, we identify that structural refactorings and concurrent combinations across branches are more strongly associated with higher merge effort, highlighting patterns that are not captured solely based on conflict occurrence.

Building upon this methodology, Mongiovi et al. [60] extended the investigation to 50 JavaScript repositories (56,966 merge scenarios, 4,816 with conflicts) using RefDiff 2.0 [62]. They found that 7% of merge scenarios have at least one refactoring within conflicting files, while 4% involved refactoring actions at the conflict region level. They observed a positive but weak correlation between these variables at both levels, and a moderate positive correlation between refactoring type diversity and merge scenarios with conflicts. Among the various refactoring types, “Internal Move” and “Move”, which encompass types such as Move Attribute, Move Variable, Move Method, Move Class, and Extract Method, were most commonly associated with merge conflicts.

Previous studies investigated the phenomena of composite refactorings [29], [34]. Composite refactorings are interlinked transformations that consists of two or refactorings (e.g., *two Extract Methods* or *one Extract Method + one Move Method*). A composite refactoring is considered complete when it fully removes the intended problem code smell [63]. Incomplete refactorings occur as often as complete refactorings [34]. The incompleteness may occur as developers aimed at reducing merge effort later. Existing studies show that developers roll back refactorings [64], [65], which may be induced by reduction of merge effort. However, those authors did not study the relationship between refactorings and merge effort.

Taken together, these prior studies highlight the importance of understanding how refactorings influence merge operations and the need to improve refactoring-merge-aware tools. However, most of these studies focus on a small subset of

refactoring types, despite the wide variety of transformations commonly observed in real-world software development [14], [34], [37], [35], [36]. Additionally, they tend to examine the relationship between specific refactorings and the occurrence of merge conflicts, with an emphasis on conflict chunk size rather than the effort required to resolve these conflicts.

By addressing these combined limitations, our work deepens the analysis by investigating not only the specific behavior of each refactoring type but also how different combinations, applied simultaneously in parallel branches, relate to merge effort. In doing so, it complements and extends previous findings, offering a more fine-grained understanding of the mechanisms through which individual refactorings and their interactions increase the complexity and cost of integrating parallel development changes.

VI. CONCLUSION

Refactorings are central to improving code quality, but can introduce merge risks that are often overlooked in collaborative development. Our results show that the presence, frequency, and diversity of certain refactoring types, particularly renamings and structural changes (e.g., move, extract, split), are associated with increased merge effort, whereas more localized modifications (e.g., parameter or variable adjustments, attribute or variable extractions) exhibit weaker associations. We also find that combinations of refactorings across branches tend to show stronger associations with merge effort, especially when involving structural transformations that reorganize elements. These findings require better means for coordination when applying high-impact refactorings, as well as tool support to detect and mitigate merge risks.

Key Lessons Learned. Our findings indicate that: (1) change volume (e.g., number of modified lines and files) provides a useful baseline but is insufficient alone, as refactoring-aware signals capture structural aspects of change not explained by churn; (2) refactoring type plays a central role, with structural transformations showing consistently stronger associations with merge effort, even at low volumes; (3) refactoring volume (TNR) and diversity (NDRT) are independent drivers of merge complexity; (4) the concurrent application of refactorings across branches is a critical risk factor, particularly when combinations modify shared data-structure representations, method signatures, or redistribute responsibilities across code elements; (5) localized transformations tend to have more limited impact on integration than architectural changes affecting class hierarchies and shared abstractions; (6) refactoring interactions amplify merge effort, even for transformations that are typically safe in isolation; (7) merge effort emerges not only from textual conflicts but from the interaction of concurrent changes, reinforcing the need for coordination strategies and refactoring-aware tool support; and (8) merge effort should be studied from a multidimensional perspective, as factors such as cognitive load and resolution time motivate further qualitative investigation for deeper understanding.

Our findings lay the groundwork for building refactoring risk models and promoting safer collaboration in parallel development workflows. Future work may include validating

these insights in industrial settings, exploring other programming languages, including additional refactoring types not explored in this study, conducting qualitative studies with developers, and improving semantic conflict detection to deepen understanding of how refactorings relate to merge effort. Additionally, future work may explore alternative effort proxies, such as fine-grained change measures based on Levenshtein edit distance [45], to complement line-based churn metrics and provide a more detailed view of code modifications. We also consider relying on eye trackers to derive measurements of cognitive effort. The analysis of eye movements while a developer is performing the merge would capture elements associated with the cognitive process. Recent studies have used eye trackers to assess the cognitive effort of developers understanding refactored code [66], [67] and opportunities for refactoring (smells) [68]. No existing work has explored the use of eye trackers on monitoring merge effort.

Finally, we consider it an important direction to investigate how AI-assisted development tools may influence merge effort, particularly in scenarios where automated support can reduce manual intervention, and how AI agents may affect the relationship between code changes and perceived effort in modern collaborative development practices. Another potential direction for future work is investigating refactorings introduced during merge commits. It would be interesting to analyze whether they primarily serve to reconcile conflicting changes or whether developers use the merge process to opportunistically improve the code.

ACKNOWLEDGMENTS

We acknowledge the use of ChatGPT 5 and Grammarly for improving the grammar, vocabulary and text style. All suggestions were carefully examined and fixed by the authors; we take full responsibility for the paper content. The authors would like to thank CNPq (grant 315711/2020-5, 309300/2023-1, 420025/2023-5, 310391/2025-3, and 308984/2025-0) and FAPERJ (grant E-26/201.139/2022, E-26/200.510/2023, E-26/211.033/2019, E-26/211.033/2019 and E-26/204.145/2024) for the financial support.

REFERENCES

- [1] G. Ghiotto, L. Murta, M. Barros, and A. van der Hoek, "On the nature of merge conflicts: A study of 2,731 open source Java projects hosted by GitHub," *IEEE Trans. Softw. Eng.*, vol. 46, no. 8, pp. 892–915, 2020.
- [2] Y. Brun, R. Holmes, M. D. Ernst, and D. Notkin, "Proactive detection of collaboration conflicts," in *Proc. ESEC/FSE*, 2011, pp. 168–178.
- [3] B. K. Kasi and A. Sarma, "Cassandra: Proactive conflict minimization through optimized task scheduling," in *Proc. ICSE*, 2013, pp. 732–741.
- [4] T. Zimmermann, "Mining workspace updates in CVS," in *Proc. MSR*, 2007, p. 11.
- [5] T. Mens, "A state-of-the-art survey on software merging," *IEEE Trans. Softw. Eng.*, vol. 28, no. 5, pp. 449–462, 2002.
- [6] S. Apel, J. Liebig, B. Brandl, C. Lengauer, and C. Kästner, "Semistructured merge: Rethinking merge in revision control systems," in *Proc. ESEC/FSE*, 2011, pp. 190–200.
- [7] H. Shen and C. Sun, "Syntax-based reconciliation for asynchronous collaborative writing," in *Proc. CollabCom*, 2005, pp. 1–10.
- [8] S. Apel, O. LeBenich, and C. Lengauer, "Structured merge with auto-tuning: balancing precision and performance," in *Proc. ASE*, 2012, pp. 120–129.
- [9] O. LeBenich, S. Apel, and C. Lengauer, "Balancing precision and performance in structured merge," *Autom. Softw. Eng.*, vol. 22, no. 3, pp. 367–397, 2015.

- [10] T. Apiwattanapong, A. Orso, and M. J. Harrold, "JDiff: A differencing technique and tool for object-oriented programs," *Autom. Softw. Eng.*, vol. 14, no. 1, pp. 3–36, 2007.
- [11] D. Albuquerque *et al.*, "Managing technical debt using intelligent techniques - a systematic mapping study," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2202–2220, 2023.
- [12] W. Charoenwet, P. Thongtanunam, V. Pham, and C. Treude, "Toward effective secure code reviews: an empirical study of security-related coding weaknesses," *Empir. Softw. Eng.*, vol. 29, no. 4, p. 88, 2024.
- [13] D. Silva, N. Tsantalis, and M. T. Valente, "Why we refactor? confessions of GitHub contributors," in *Proc. FSE*, 2015, pp. 858–870.
- [14] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018.
- [15] J. Pantiuchina *et al.*, "Why developers refactor source code: A mining-based study," *ACM Trans. Softw. Eng. Methodol.*, vol. 29, no. 4, 2020.
- [16] D. Dig, K. Manzoor, R. Johnson, and T. N. Nguyen, "Refactoring-aware configuration management for object-oriented programs," in *Proc. ICSE*, 2007, pp. 427–436.
- [17] O. LeBenich, S. Apel, C. Kästner, G. Seibt, and J. Siegmund, "Renaming and shifted code in structured merging: Looking ahead for precision and performance," in *Proc. ASE*, 2017, pp. 543–553.
- [18] M. Mahmoudi and S. Nadi, "The Android update problem: An empirical study," in *Proc. MSR*, 2018, pp. 220–230.
- [19] A. László, L. Lengyel, and H. Charaf, "Detecting renamings in three-way merging," *Acta Polytech. Hung.*, vol. 4, no. 4, 2007.
- [20] M. Mahmoudi, S. Nadi, and N. Tsantalis, "Are refactorings to blame? an empirical study of refactorings in merge conflicts," in *Proc. SANER*, 2019, pp. 151–162.
- [21] J. Han, J. Pei, and H. Tong, *Data Mining: Concepts and Techniques*, 4th ed. Morgan Kaufmann, 2022.
- [22] D. I. K. Sjøberg, A. Yamashita, B. C. D. Anda, A. Mockus, and T. Dybå, "Quantifying the effect of code smells on maintenance effort," *IEEE Trans. Softw. Eng.*, vol. 39, no. 8, pp. 1144–1156, 2012.
- [23] A. Oliveira, V. Neves, A. Plastino, A. C. Bibiano, A. Garcia, and L. Murta, "Do code refactorings influence the merge effort?" in *Proc. ICSE*, 2023, pp. 134–146.
- [24] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proc. MSR*, 2014, pp. 92–101.
- [25] H. Borges and M. T. Valente, "What's in a GitHub star? understanding repository starring practices in a social coding platform," *J. Syst. Softw.*, vol. 146, pp. 112–129, 2018.
- [26] TIOBE, "TIOBE index for december 2024," <https://www.tiobe.com/tiobe-index/>, 2024.
- [27] StackOverflow, "Developer survey results," <https://survey.stackoverflow.co/2024/>, 2024.
- [28] V. Barnett and T. Lewis, *Outliers in Statistical Data*, 3rd ed. Wiley, 1994.
- [29] A. C. Bibiano *et al.*, "Enhancing recommendations of composite refactorings based on the practice," in *Proc. SCAM*, 2024, pp. 83–93.
- [30] G. Vale *et al.*, "Predicting merge conflicts considering social and technical assets," *Empir. Softw. Eng.*, vol. 29, no. 1, p. 24, 2024.
- [31] I. Ferreira *et al.*, "Assessing the bug-proneness of refactored code: A longitudinal multi-project study," in *Proc. EASE'25*, 2025, pp. 79–90.
- [32] P. Alikhanifard and N. Tsantalis, "A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, 2025.
- [33] N. Tsantalis, A. Ketkar, and D. Dig, "RefactoringMiner 2.0," *IEEE Trans. Softw. Eng.*, vol. 48, no. 3, pp. 930–950, 2020.
- [34] A. C. Bibiano *et al.*, "Look ahead! revealing complete composite refactorings and their effects," in *Proc. ICSME'21*, 2021, pp. 298–308.
- [35] B. Liu *et al.*, "An automated approach to discovering software refactorings by comparing successive versions," *IEEE Trans. Softw. Eng.*, vol. 51, no. 5, pp. 1358–1380, 2025.
- [36] S. Wang *et al.*, "Refactoring deep learning code: A study of practices and unsatisfied tool needs," in *Proc. ICSME*, 2025, pp. 1–12.
- [37] A. Alameer and A. Alazba, "Predicting software refactoring opportunities using two-level TB-Stacking ensemble models," *Autom. Softw. Eng.*, vol. 33, no. 1, p. 32, 2026.
- [38] J. G. Prudêncio, L. Murta, C. Werner, and R. Cepêda, "To lock, or not to lock: That is the question," *J. Syst. Softw.*, vol. 85, no. 2, pp. 277–289, 2012.
- [39] T. Silva and L. Murta, "Uma técnica para a quantificação do esforço de merge," in *Proc. VEM*, 2018.
- [40] L. D. Silva *et al.*, "Detecting semantic conflicts with unit tests," *J. Syst. Softw.*, vol. 214, p. 112070, 2024.
- [41] T. Olsson, M. Ericsson, and A. Wingkvist, "The relationship of code churn and architectural violations in the open source software JabRef," in *Proc. ECSA-Companion*, 2017, pp. 152–158.
- [42] J. Çarka, M. Esposito, and D. Falessi, "On effort-aware metrics for defect prediction," *Empir. Softw. Eng.*, vol. 27, no. 6, p. 152, 2022.
- [43] K. Jesse *et al.*, "RefactorScore: Evaluating refactor prone code," *IEEE Trans. Softw. Eng.*, vol. 49, no. 11, pp. 5008–5026, 2023.
- [44] N. Bessghaier *et al.*, "Towards understanding code review practices for infrastructure-as-code: An empirical study on OpenStack projects," *Empir. Softw. Eng.*, vol. 30, no. 4, p. 106, 2025.
- [45] L. Yujian and B. Liu, "A normalized Levenshtein distance metric," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 29, no. 6, pp. 1091–1095, 2007.
- [46] S. Chowdhury, R. Holmes, A. Zaidman, and R. Kazman, "Revisiting the debate: Are code metrics useful for measuring maintenance effort?" *Empir. Softw. Eng.*, vol. 27, no. 6, p. 158, 2022.
- [47] Aguado-Puig *et al.*, "Accelerating edit-distance sequence alignment on gpu using the wavefront algorithm," *IEEE Access*, vol. 10, pp. 63 782–63 796, 2022.
- [48] I. H. Witten, E. Frank, and M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques*, 3rd ed. Elsevier, 2016.
- [49] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules in large databases," in *Proc. VLDB*, 1994, pp. 487–499.
- [50] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in *Proc. SIGMOD*, 2000, pp. 1–12.
- [51] M. J. Zaki, "Scalable algorithms for association mining," *IEEE Trans. Knowl. Data Eng.*, vol. 12, no. 3, pp. 372–390, 2000.
- [52] C. Spearman, "The proof and measurement of association between two things," *Amer. J. Psychol.*, vol. 15, no. 1, pp. 72–101, 1904.
- [53] R. M. O'Brien, "A caution regarding rules of thumb for variance inflation factors," *Qual. Quant.*, vol. 41, no. 5, pp. 673–690, 2007.
- [54] P. Hammant, "What is trunk-based development?" <https://paulhammant.com/2013/04/05/>, 2013.
- [55] R. Mahdavi-Hezaveh, J. Dremann, and L. A. Williams, "Feature toggle driven development: Practices used by practitioners," *CoRR*, vol. abs/1907.06157, 2019.
- [56] M. C. Lud and G. Widmer, "Relative unsupervised discretization for association rule mining," in *Proc. PKDD*, 2000, pp. 148–158.
- [57] D. Li *et al.*, "A new approach of self-adaptive discretization to enhance the apriori quantitative association rule mining," in *Proc. ISDEA*, 2012, pp. 44–47.
- [58] S. C. Tan, "Improving association rule mining using clustering-based discretization of numerical data," in *Proc. ICONIC*, 2018, pp. 1–5.
- [59] M. Ellis, S. Nadi, and D. Dig, "Operation-based refactoring-aware merging: An empirical evaluation," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2698–2721, 2023.
- [60] M. Mongiovi, J. G. Oliveira, and S. Souto, "An empirical study of the relationship between refactorings and merge conflicts in Javascript code," in *Proc. SBES*, 2023, pp. 89–98.
- [61] T. Morse, "CVS," *Linux J.*, vol. 1996, no. 21es, 1996.
- [62] D. Silva, J. P. da Silva, G. S. island R. Terra, and M. T. Valente, "RefDiff 2.0: A multi-language refactoring detection tool," *IEEE Trans. Softw. Eng.*, vol. 47, no. 12, pp. 2786–2802, 2020.
- [63] A. C. Bibiano *et al.*, "How does incomplete composite refactoring affect internal quality attributes?" in *Proc. ICPC'20*, 2020, pp. 149–159.
- [64] M. Paixão *et al.*, "Behind the intents: An in-depth study on software refactoring in modern code review," in *Proc. MSR'20*, 2020, pp. 125–136.
- [65] A. Uchôa *et al.*, "How does modern code review impact software design degradation? an in-depth study," in *Proc. ICSME'20*, 2020, pp. 511–522.
- [66] J. A. S. da Costa *et al.*, "Refactoring for novices in java: An eye tracking study on the extract vs. inline methods," *J. Syst. Softw.*, vol. 237, 2026.
- [67] —, "Evaluating refactorings for disciplined #ifdef annotations: An eye tracking study with novices," *Empir. Softw. Eng.*, vol. 26:5, 2021.
- [68] V. Martins *et al.*, "Eyes on code smells: Analyzing developers' responses during code snippet analysis," in *Proc. SBES'24*, 2024, pp. 302–312.